

Clases

Python para ciencias de la atmósfera

Lucía Pedraza Díaz

creamos una
clase con class



```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

Nombre de la clase.
Convención: primer
letra de cada palabra
con mayúscula.



```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

Herencia



```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

← docstring

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```



```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

__init__ es el inicializador. Se llama al crear la instancia.

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

self es la instancia misma. Va explícito como primer argumento de cada método de instancia. Es convención, no palabra reservada.

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

Los parámetros del inicializador son **los datos que la clase exige para crear una instancia**. Cuando generas un objeto a partir de la clase, le pasas esos valores, y el inicializador los usa para crear los atributos de esa instancia en particular.

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

Atributos de la instancia:

Un atributo es un valor que pertenece a un objeto y al que accedes con punto: `objeto.nombre`

Los atributos de instancia pertenecen a un objeto en particular.

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

Un **método** es una función definida dentro de una clase. Se escribe igual que cualquier función

```
class NombreClase(ClaseBase):  
    """Docstring opcional."""  
  
    atributo_de_clase = 0 # compartido por todas las instancias  
  
    def __init__(self, param1, param2):  
        self.param1 = param1 # atributo de instancia  
        self.param2 = param2  
  
    def metodo(self, x):  
        return self.param1 + x
```

```
instancia1 = NombreClase(10,5)  
print(instancia1.atributo_de_clase)  
print('param1: ', instancia1.param1)  
print(instancia1.param2)
```

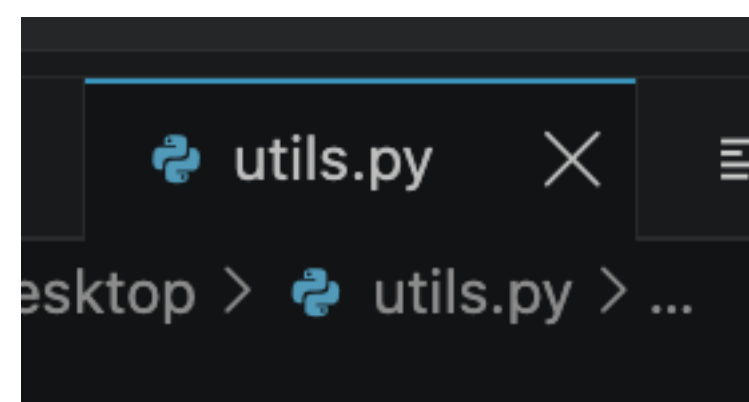
Una instancia es un objeto creado a partir de una clase. De una clase puedes crear muchos objetos, cada uno con atributos diferentes.

¿Qué es un módulo?

- Un módulo es un archivo .py
- Agrupa código relacionado (funciones, clases, variables) para poder reutilizarlo e importarlo desde otros archivos.
- Módulo $\neq$ paquete (o librería)
- El módulo es un archivo individual.
- Un paquete son varios módulos en una carpeta.
- Librería: un paquete o conjunto de paquetes que instalas.

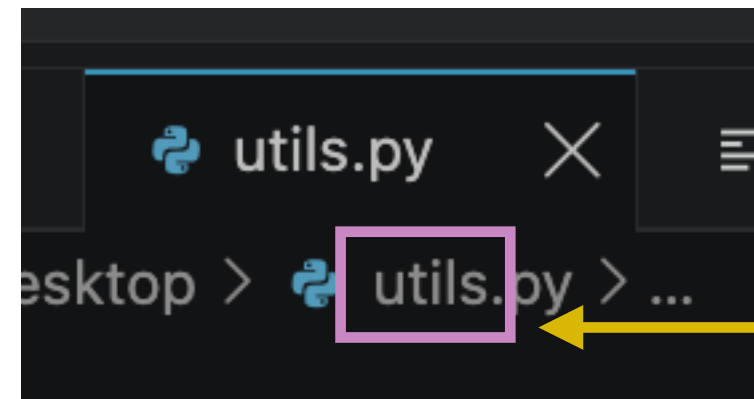
<https://docs.python.org/3/tutorial/modules.html>

Nombre del módulo



```
def suma(*args):  
    print(f'La suma es {sum(args)}')  
  
def minimo(*args):  
    print(f'El mínimo es {min(args)}')  
  
t = (3,5,6)  
suma(*t)  
minimo(*t)
```

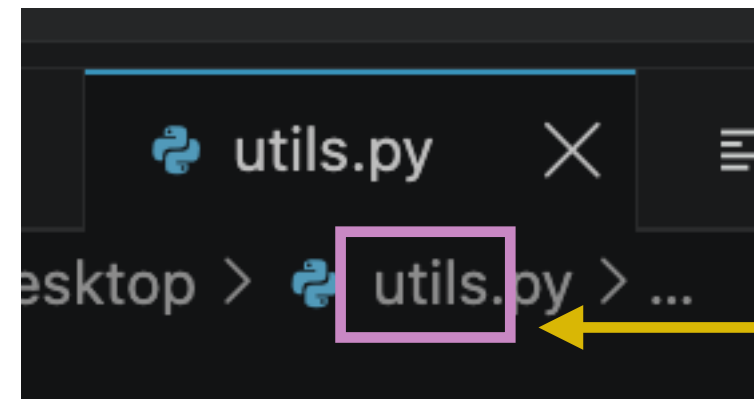
Nombre del módulo



El nombre del módulo es el nombre del archivo, sin el .py

```
def suma(*args):  
    print(f'La suma es {sum(args)}')  
  
def minimo(*args):  
    print(f'El mínimo es {min(args)}')  
  
t = (3,5,6)  
suma(*t)  
minimo(*t)
```

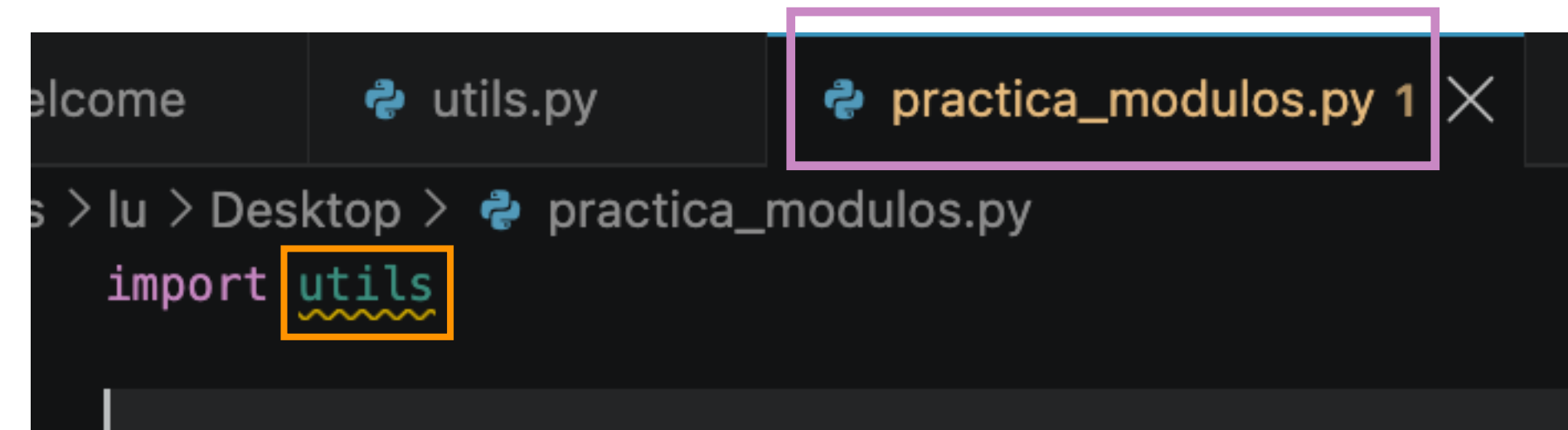
Nombre del módulo



El nombre del módulo es el nombre del archivo, sin el .py

```
def suma(*args):  
    print(f'La suma es {sum(args)}')  
  
def minimo(*args):  
    print(f'El mínimo es {min(args)}')  
  
t = (3,5,6)  
suma(*t)  
minimo(*t)
```

Para importar un módulo usamos la palabra reservada **import**



Espacio de nombres global

```
import utils  
  
print(globals())
```

```
La suma es 14  
El mínimo es 3  
{'__name__': '__main__', '__doc__': None, '__package__': None,  
 '__loader__': <_frozen_importlib_external.SourceFileLoader object  
at 0x102e23ca0>, '__spec__': None, '__annotations__': {},  
 '__builtins__': <module 'builtins' (built-in)>, '__file__': '/  
Users/lu/Desktop/practica_modulos.py', '__cached__': None,  
'utils': <module 'utils' from '/Users/lu/Desktop/utils.py'>}
```

Espacio de nombres global

```
import utils  
  
print(globals())
```

```
La suma es 14  
El mínimo es 3
```

```
{'__name__': '__main__', '__doc__': None, '__package__': None,  
 '__loader__': <_frozen_importlib_external.SourceFileLoader object  
at 0x102e23ca0>, '__spec__': None, '__annotations__': {},  
 '__builtins__': <module 'builtins' (built-in)>, '__file__': '/  
Users/lu/Desktop/practica_modulos.py', '__cached__': None,  
 'utils': <module 'utils' from '/Users/lu/Desktop/utils.py'>}
```

Importante: notar que se ejecuta utils.py

Import

```
1  import utils
2
3  utils.suma(1,5,6)
4  utils.minimo(1,5,6)
```

OUTPUT ... Code

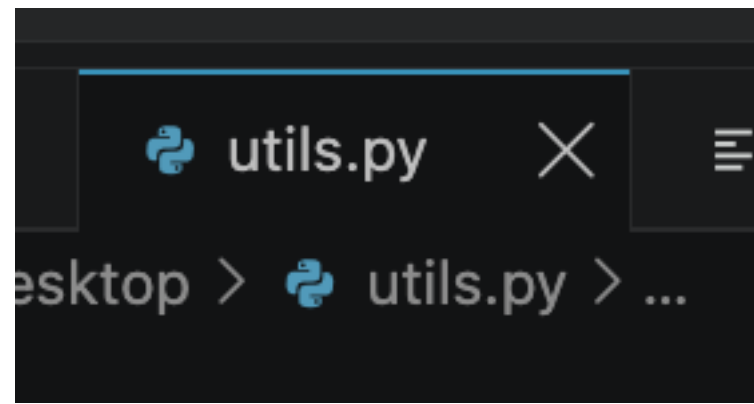
Filter (e.g. text, !excludeText, text1,text2)

La suma es 14
El mínimo es 3
La suma es 12
El mínimo es 1

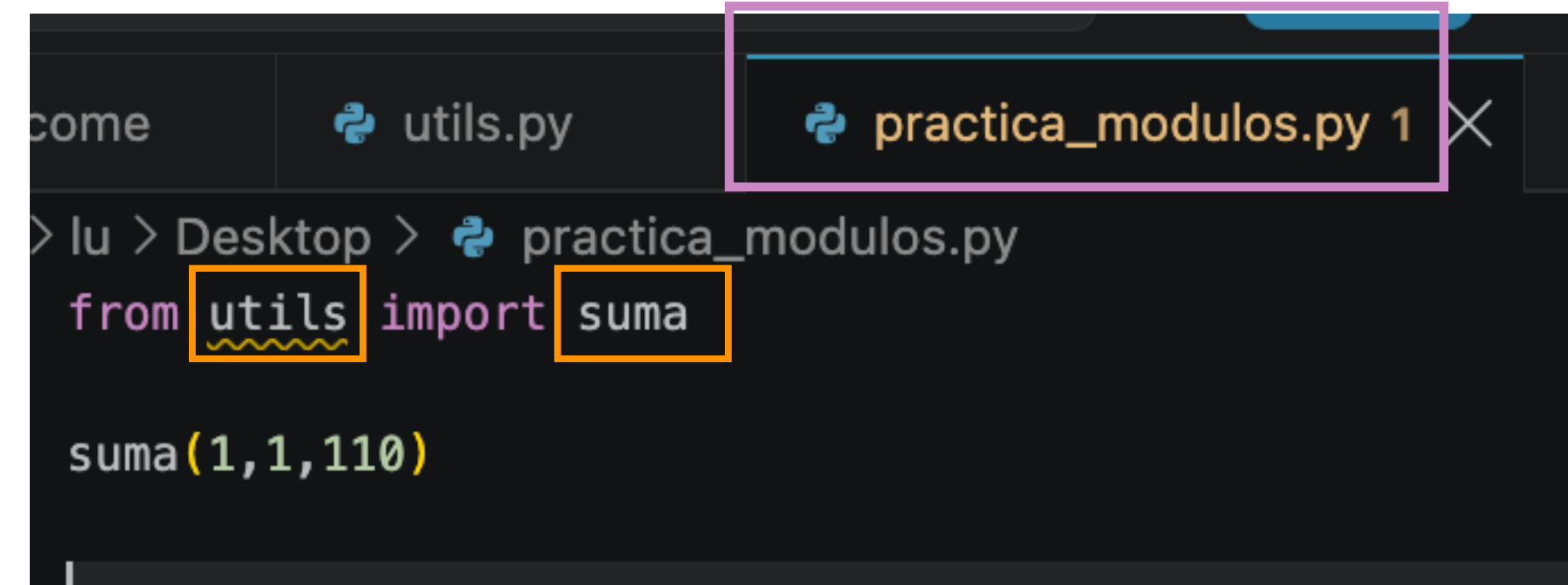
Para acceder a la función suma, escribimos el nombre del módulo, seguido de un punto y el nombre del objeto (función suma en este caso).

Import

Con **from** podemos decidir que importar



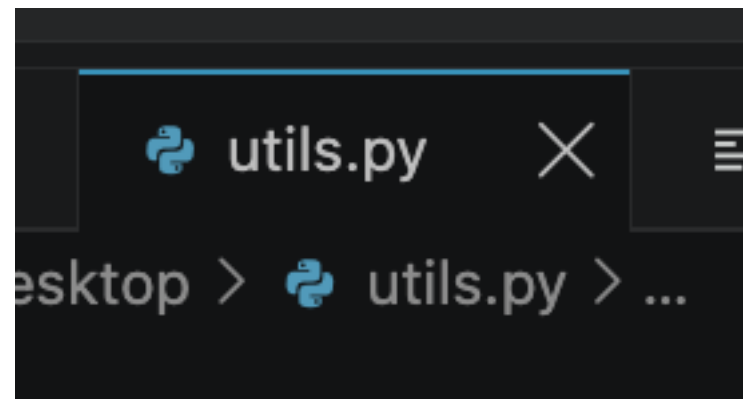
```
def suma(*args):  
    print(f'La suma es {sum(args)}')  
  
def minimo(*args):  
    print(f'El mínimo es {min(args)}')  
  
t = (3,5,6)  
suma(*t)  
minimo(*t)
```



```
from utils import suma  
  
suma(1,1,110)
```

*se pueden importar funciones, clases, variables, etc

Import

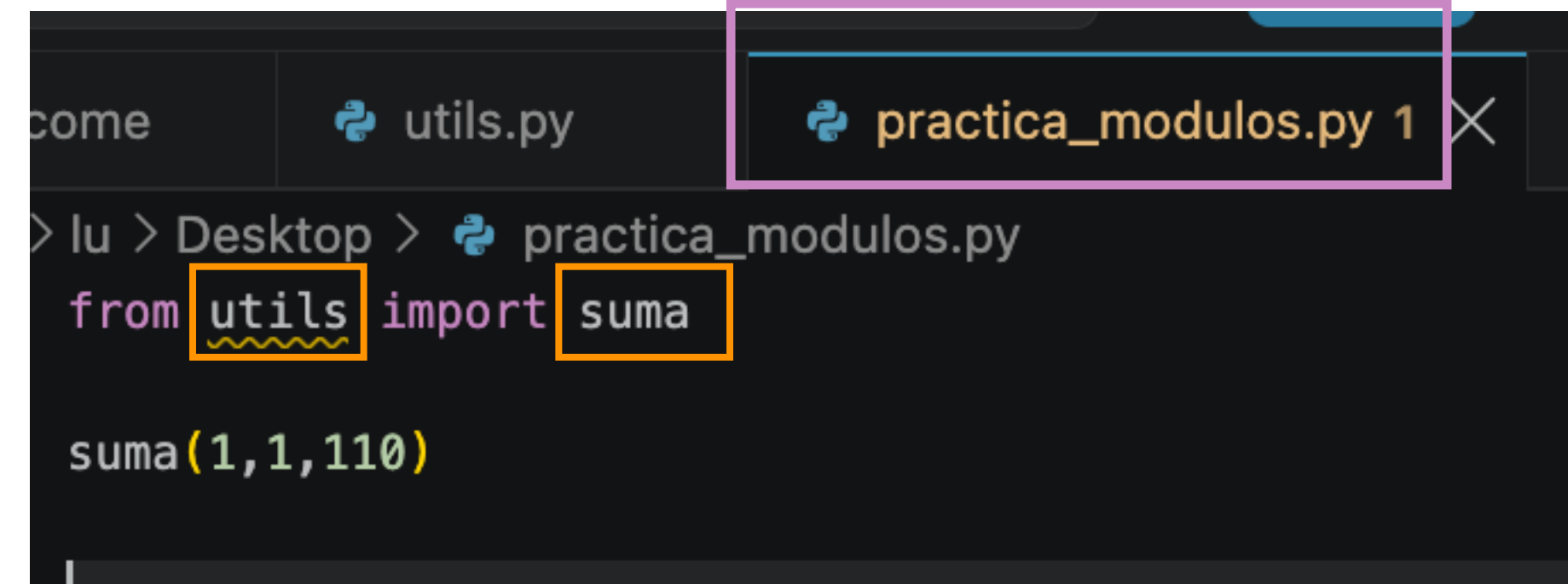


```
def suma(*args):
    print(f'La suma es {sum(args)}')

def minimo(*args):
    print(f'El mínimo es {min(args)}')

t = (3,5,6)
suma(*t)
minimo(*t)
```

Con **from** podemos decidir que importar



*se pueden importar funciones, clases, variables, etc

Importamos la función minimo:

```
from utils import suma, minimo

suma(1,1,110)
minimo(1,1,100)
```

Import

```
from utils import suma, minimo  
  
print(globals())
```

Los módulos importados se añaden al espacio de nombre global

```
La suma es 14  
El mínimo es 3  
{'__name__': '__main__', '__doc__': None, '__package__': None,  
  '__loader__': <_frozen_importlib_external.SourceFileLoader object  
at 0x109dc6ca0>, '__spec__': None, '__annotations__': {},  
  '__builtins__': <module 'builtins' (built-in)>, '__file__': '/  
Users/lu/Desktop/practica_modulos.py', '__cached__': None, 'suma':  
<function suma at 0x109fe0940>, 'minimo': <function minimo at  
0x109fe0af0>}
```

Import

Podemos importar todo con *

```
from utils import *  
  
suma(1,1,110)  
minimo(1,1,100)
```

Nota: no se recomienda importar todo.

Import as

Podemos ponerle un apodo al módulo

```
1  import utils as funciones
2
3  funciones.suma(1,5,6)
4  funciones.minimo(1,5,6)
```

OUTPUT ...

Code

Filter (e.g. text, !excludeText, text1,text2)

```
la suma es 14
el mínimo es 3
la suma es 12
el mínimo es 1
```

as: pone un apodo al nombre que estamos importando

La idea es acortar nombres largos. Ejemplos:

- `import numpy as np`
- `import pandas as pd`
- `import matplotlib.pyplot as plt`

Import as

Podemos ponerle apodo a los objetos del módulo

```
from utils import suma as s
from utils import minimo as m

s(1,5,6)
m(1,5,6)
```

La variable `__name__`

Cada vez que python carga un módulo, crea automáticamente una variable especial llamada `__name__` y le asigna un valor.

```
utils.py  ✕  practica_modulos.py
> lu > Desktop > utils.py > ...

def suma(*args):
    print(f'La suma es {sum(args)}')

def minimo(*args):
    print(f'El mínimo es {min(args)}')

t = (3,5,6)
suma(*t)
minimo(*t)

print(f'Nombre del módulo: {__name__}')
```

```
utils.py  practica_modulos.py 2 ✕
s > lu > Desktop > practica_modulos.py

from utils import suma as s
from utils import minimo as m

s(1,5,6)
m(1,5,6)

print(f'Nombre del módulo: {__name__}')
```

La variable `__name__`

Cada vez que python carga un módulo, crea automáticamente una variable especial llamada `__name__` y le asigna un valor.

```
utils.py  X  practica_modulos.py
> lu > Desktop > utils.py > ...

def suma(*args):
    print(f'La suma es {sum(args)}')

def minimo(*args):
    print(f'El mínimo es {min(args)}')

t = (3,5,6)
suma(*t)
minimo(*t)

print(f'Nombre del módulo: {__name__}')
```

```
utils.py  practica_modulos.py 2 X
> lu > Desktop > practica_modulos.py

from utils import suma as s
from utils import minimo as m

s(1,5,6)
m(1,5,6)

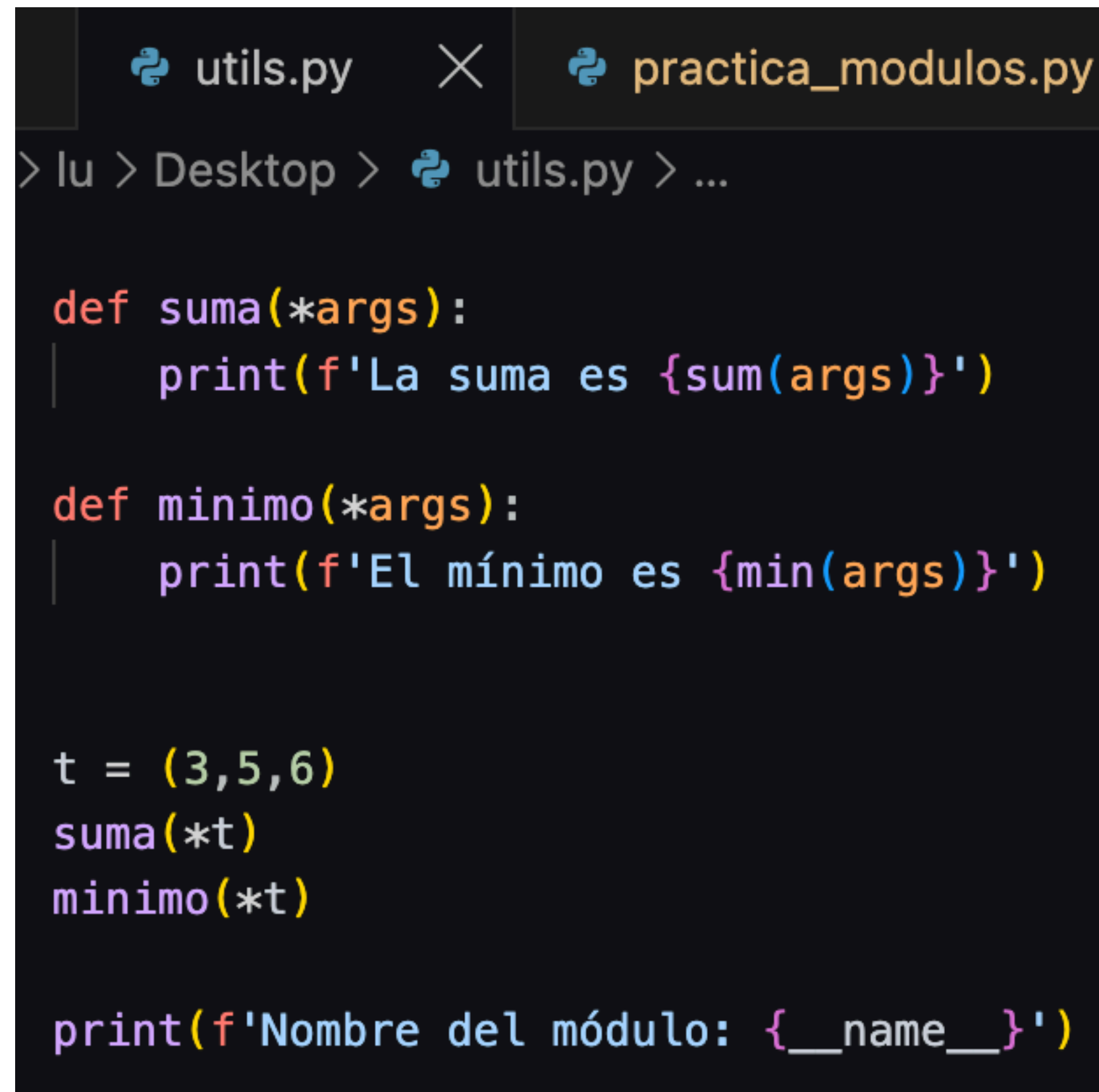
print(f'Nombre del módulo: {__name__}')
```

Si el archivo se ejecuta directamente:

`__name__ = __main__`

La variable `__name__`

Cada vez que python carga un módulo, crea automáticamente una variable especial llamada `__name__` y le asigna un valor.



```

> lu > Desktop > utils.py > ...

def suma(*args):
    print(f'La suma es {sum(args)}')

def minimo(*args):
    print(f'El mínimo es {min(args)}')

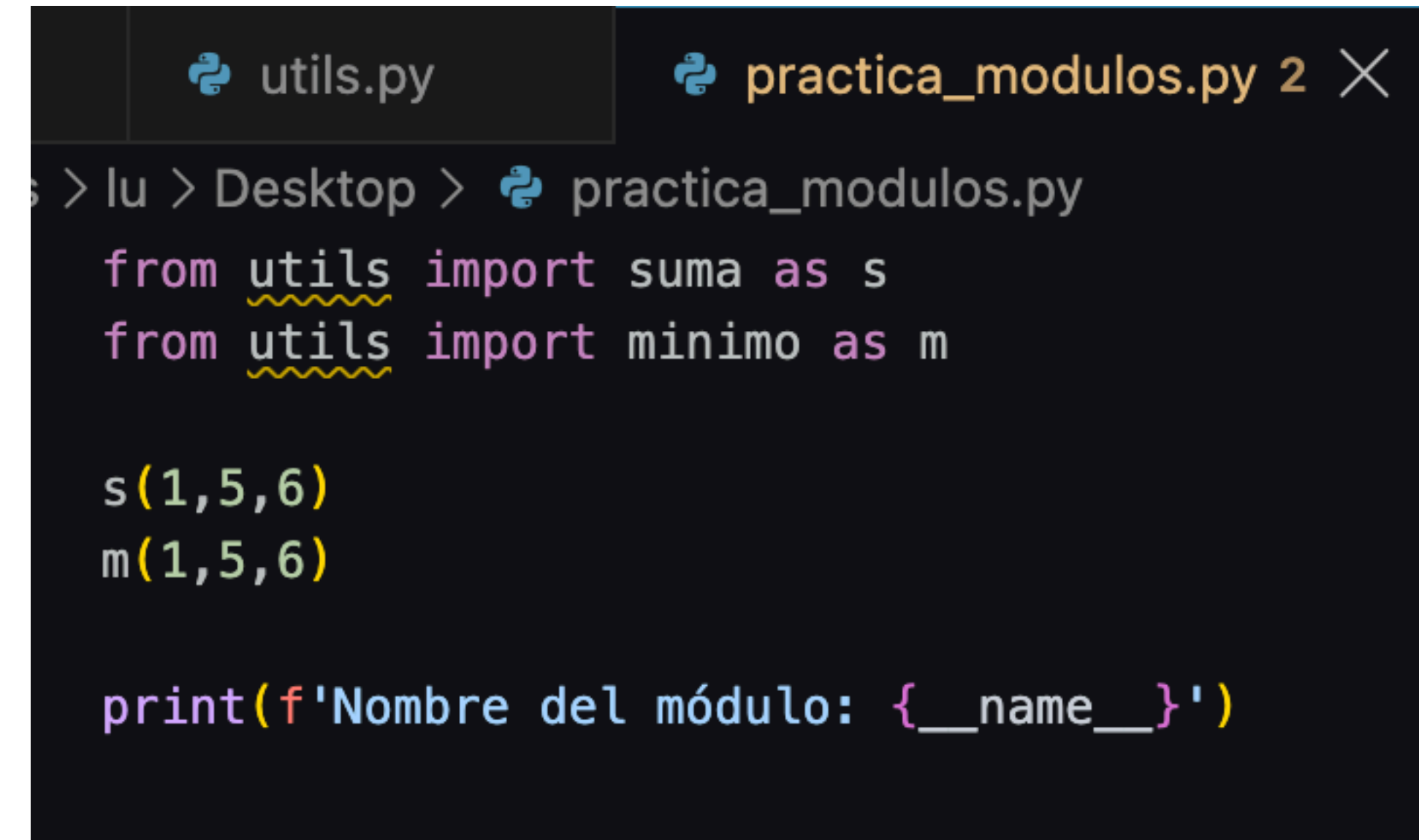
t = (3,5,6)
suma(*t)
minimo(*t)

print(f'Nombre del módulo: {__name__}')

```

Si el archivo se ejecuta directamente:

`__name__ = __main__`



```

> lu > Desktop > practica_modulos.py

from utils import suma as s
from utils import minimo as m

s(1,5,6)
m(1,5,6)

print(f'Nombre del módulo: {__name__}')

```

Si el archivo se importa desde otro:

`__name__ = <nombre del módulo>`

La variable `__name__`

```
utils.py  X  practica_modulos.py
> lu > Desktop > utils.py > ...

def suma(*args):
    print(f'La suma es {sum(args)}')

def minimo(*args):
    print(f'El mínimo es {min(args)}')

t = (3,5,6)
suma(*t)
minimo(*t)

print(f'Nombre del módulo: {__name__}')
```



```
utils.py  practica_modulos.py 2 X
> lu > Desktop > practica_modulos.py

from utils import suma as s
from utils import minimo as m

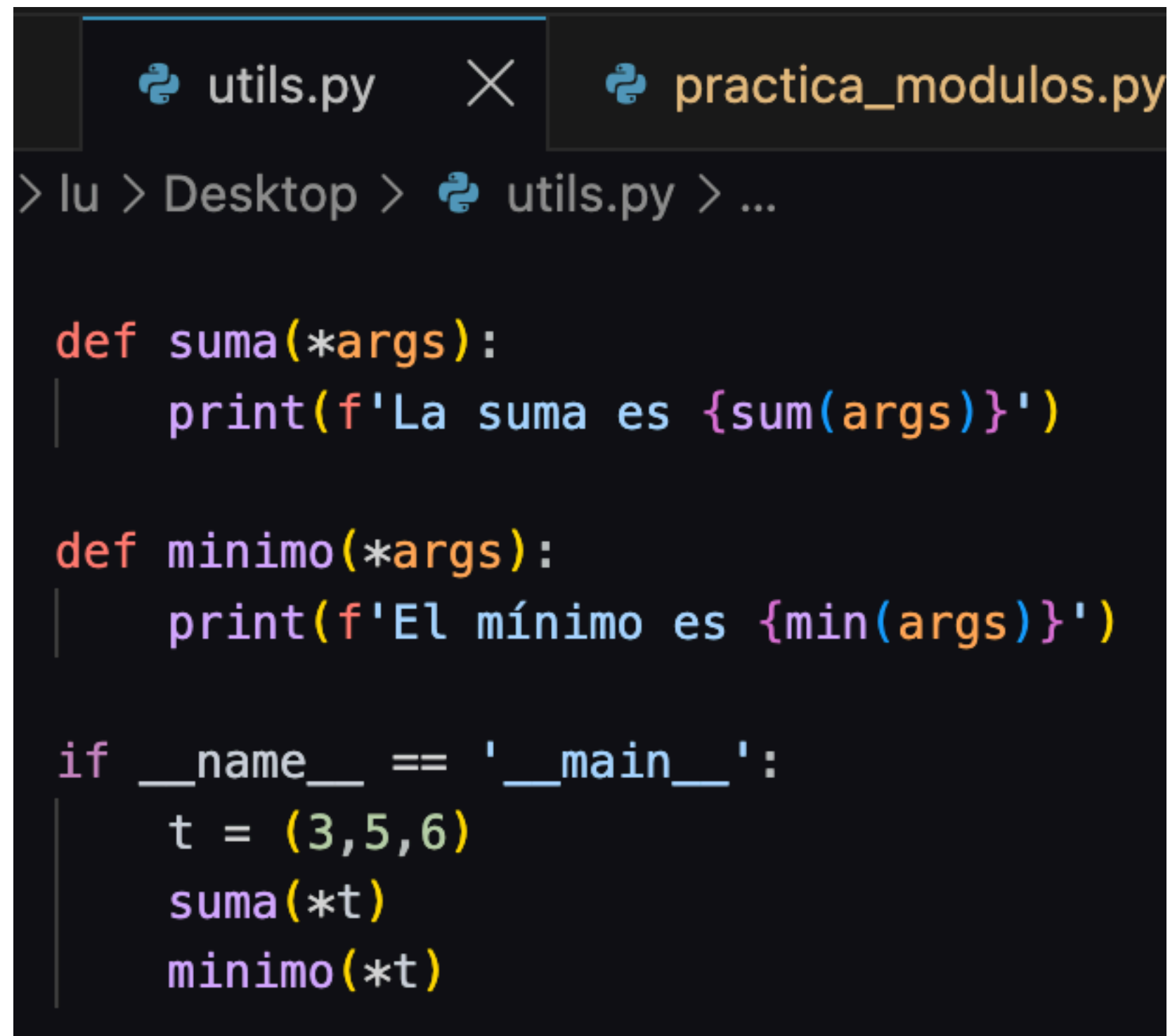
s(1,5,6)
m(1,5,6)

print(f'Nombre del módulo: {__name__}')
```

Cuando importamos un módulo python ejecuta todo el contenido del módulo.

Hay cosas que no queremos que se ejecuten o queremos que solo se ejecuten cuando se corre directamente el módulo

La variable `__name__`

A screenshot of a code editor with two tabs: 'utils.py' and 'practica_modulos.py'. The active tab is 'practica_modulos.py'. The code in the editor is as follows:

```
> lu > Desktop > utils.py > ...  
  
def suma(*args):  
    print(f'La suma es {sum(args)}')  
  
def minimo(*args):  
    print(f'El mínimo es {min(args)}')  
  
if __name__ == '__main__':  
    t = (3,5,6)  
    suma(*t)  
    minimo(*t)
```

Usamos un `if` para separar el código que solo debe correr al ejecutarse el archivo directamente.

- * buena práctica para archivos que quieres importar.
- * Si es un archivo de puras funciones no es necesario.

Import

```
Desktop  
Desktop mkdir otra_carpeta  
Desktop  
Desktop  
Desktop mv utils.py otra_carpeta  
Desktop
```

¿Qué pasa si mis módulos no están en la misma carpeta?

Import

```
Desktop
Desktop mkdir otra_carpeta
Desktop
Desktop
Desktop mv utils.py otra_carpeta
Desktop
```

¿Qué pasa si mis módulos no están en la misma carpeta?

```
Welcome  practica_modulos.py 1 X
Users > lu > Desktop > practica_modulos.py
1  from otra_carpeta import utils
2
3  utils.suma(1,5,6)
4  utils.minimo(1,5,6)
5
```

```
from otra_carpeta.utils import suma, minimo

suma(1,5,6)
minimo(1,5,6)
```

```

from otra_carpeta.utils import suma, minimo
import sys

print(sys.path)      # lista todas las carpetas donde
                      # busca módulos
suma(1,5,6)
minimo(1,5,6)

```

```

import sys
sys.path.append('otra_carpeta') # metemos la carpeta a la
                                # lista donde python busca
from utils import suma, minimo
suma(1,5,6)
minimo(1,5,6)

```

```

○ → otra_carpeta
● → otra_carpeta touch __init__.py
● → otra_carpeta ls
  __init__.py __pycache__ utils.py
○ → otra_carpeta

```

Paquetes

```
from otra_carpeta.utils import suma, minimo
import sys

print(sys.path)      # lista todas las carpetas donde
|_|_|_|_|_|_|_|_|    # busca módulos
suma(1,5,6)
minimo(1,5,6)
```

```
import sys
sys.path.append('otra_carpeta') # metemos la carpeta a la
|_|_|_|_|_|_|_|_|_|_|_|_|    # lista donde python busca
from utils import suma, minimo
suma(1,5,6)
minimo(1,5,6)
```

```
○ → otra_carpeta
● → otra_carpeta touch __init__.py
● → otra_carpeta ls
  __init__.py __pycache__ utils.py
○ → otra_carpeta
```

- * Para que una carpeta sea paquete hay que agregar el archivo `__init__.py` dentro de la carpeta.

Argparse

Python para ciencias de la atmósfera

Lucía Pedraza Díaz

Argparse

- Es un módulo.
- Permite que un script reciba valores desde la terminal al momento de ejecutarlo, en vez de tener los valores fijos dentro del código.

```
→ Desktop python parser_saludo.py --nombre 'Claudia'  
Hola Claudia
```

Módulo



```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

Clase



```
import argparse

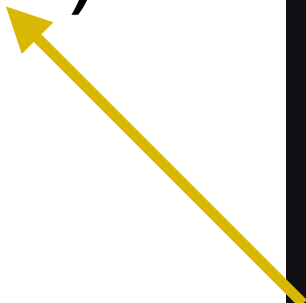
# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

Objeto que se crea a
partir de la clase
(instancia)



```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

Método

* podemos
revisar los
métodos
disponibles con
`print(dir(parser))`

```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

Le dice a parser
que valores
pedir

```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

```
→ Desktop python parser_saludo.py --nombre 'Claudia'
Hola Claudia
```

Método: lee lo
que el usuario
escribió en la
terminal y lo
parsea (lo analiza)

```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

```
→ Desktop python parser_saludo.py --nombre 'Claudia'
Hola Claudia
```

Devuelve un Namespace que contiene todos los valores ya procesados

```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

```
args = parser.parse_args()
print(args)
```

```
Namespace(nombre='Carlos')
```

```
import argparse

# creamos el parser
parser = argparse.ArgumentParser()

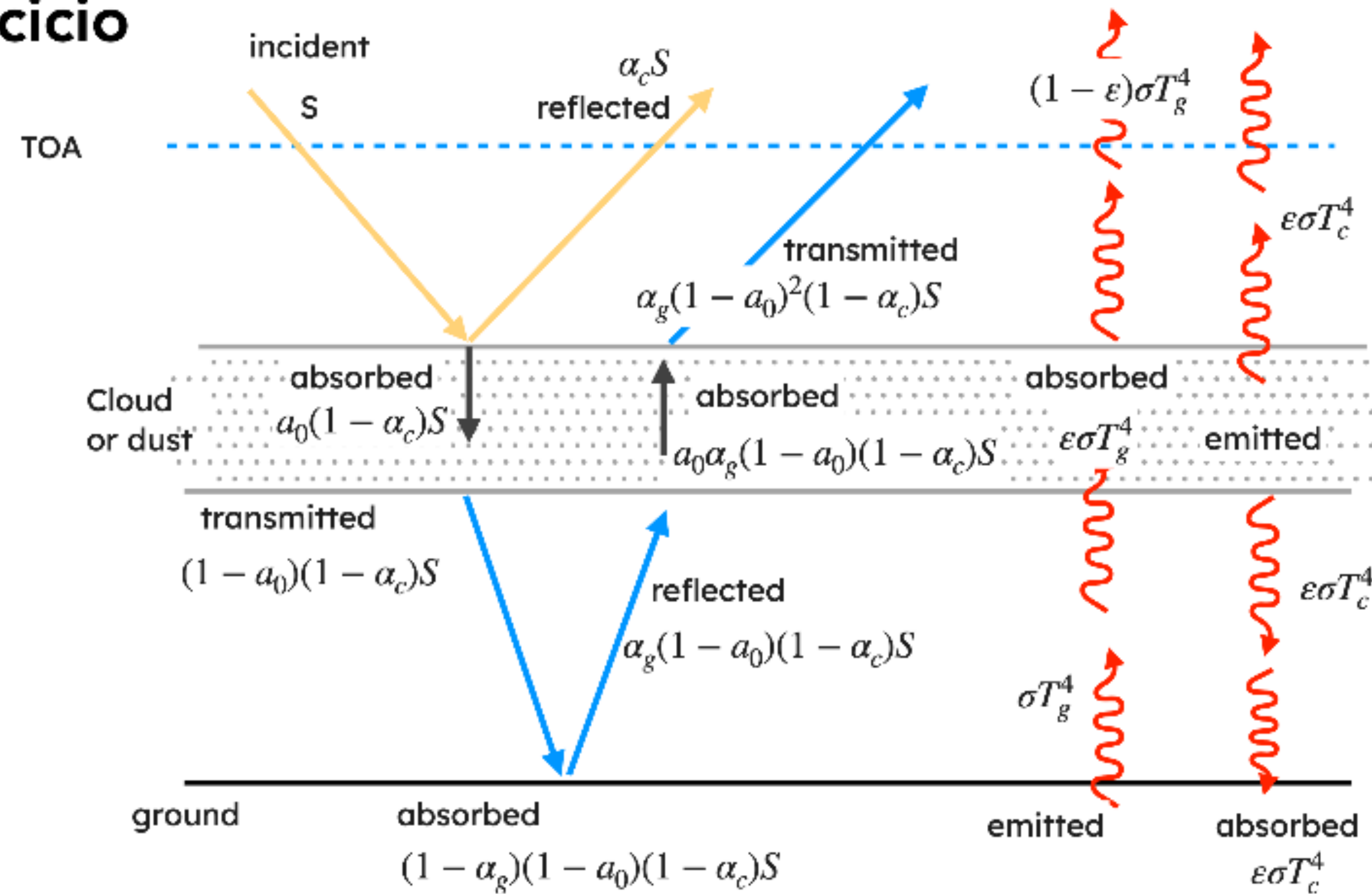
# definimos qué argumentos espera
parser.add_argument('--nombre', default = 'Carlos')

args = parser.parse_args()

print(f'Hola {args.nombre}')
```

Ejercicio

Ejercicio



la absorción infrarroja de la nube es igual a la emisividad infrarroja de la nube, ϵ

$$\sigma T_g^4 = \frac{S(1 - \alpha_c)(2 - a_0)}{(2 - \varepsilon)} \left[1 - \alpha_g(1 - a_0) \right] \quad T_c^4 = \frac{1}{\varepsilon} \left[T_g^4 - \frac{(1 - \alpha_g)(1 - a_0)(1 - \alpha_c)S}{\sigma} \right]$$

Ejemplos

Idea:

- Mostrar que las nubes pueden calentar o enfriar dependiendo del balance entre su albedo (refleja radiación solar, enfría) y su emisividad infrarroja (atrapa calor, calienta):
 - Sin nubes (referencia)
 - Aerosol volcánico (enfria)
 - Nube de agua con albedo moderado (calienta)
 - Nube de agua con albedo alto (enfria)

Caso 1: Atmósfera sin nubes

- Solo dispersión molecular (Rayleigh) y absorción atmosférica, sin nube ni aerosol.

$\alpha_c = 0.08$, $a_0 = 0.15$, $\varepsilon = 0.4$. (α_c apropiado para la dispersión solo por moléculas atmosféricas)

Caso 2: Aerosol volcánico

- Se inserta un aerosol volcánico en la atmósfera. Aumenta ligeramente el albedo, la absorción solar y la emisividad infrarroja.

$\alpha_c = 0.12$, $a_0 = 0.18$, $\varepsilon = 0.43$.

Caso 3: Nube de agua, albedo moderado

- Se introduce una nube de agua con cobertura similar a la actual. El albedo sube mucho, pero la emisividad infrarroja sube aún más.

$\alpha_c = 0.30$, $a_0 = 0.20$, $\varepsilon = 0.90$.

Caso 4: Nube de agua, albedo alto

- Igual que el caso 3 pero con albedo de nube más alto.

$\alpha_c = 0.40$, $a_0 = 0.20$, $\varepsilon = 0.90$.

```
# variables
a_0 = 0.15
alpha_c = 0.08
alpha_g = 0.10
epsilon = 0.4
sigma = 5.67e-8
S = 343

# temperatura de la superficie
num1 = S*(1-alpha_c)*(2-a_0)*(1-alpha_g*(1-a_0))
T_g4 = num1/(sigma*(2-epsilon))
T_g = T_g4**(1/4)

# temperatura de la capa
num2 = (1-alpha_g)*(1-a_0)*(1-alpha_c)*S
T_c4 = (1/epsilon)*(T_g4 - num2/sigma)
T_c = T_c4**(1/4)

print(f'T_g:{T_g:.2f} K')
print(f'T_c:{T_c:.2f} K')
```


Tarea/Exposición

- datetime
- csv
- pathlib
- json